

CSE3016 - AWS SOLUTION ARCHITECT

Dr Jasmine Selvakumari Jeya I

Senior Associate Professor Grade 2/SCOPE

VIT Bhopal University

MODULE III

Networking & Content Delivery – Amazon API Gateway, Amazon CloudFront, Direct Connect, AWS Networking and Content Delivery, Amazon VPC (Virtual Private Cloud), VPC Peering.

Amazon API Gateway

What is Amazon API Gateway?

- Amazon API Gateway is a fully managed AWS service used to create, publish, monitor, and secure APIs at any scale.
- It serves as the gateway between client applications and backend services, routing requests efficiently.
- It simplifies API management by reducing infrastructure overhead and automatically scaling with demand.
- Supports RESTful APIs, WebSocket APIs, and serverless applications with flexible and cost-effective options.

Key Features of Amazon API Gateway

- **Traffic Management**

- Provides load balancing, request throttling, and caching for optimal API performance under heavy traffic.

- **Security Features**

- Offers authentication and authorization through IAM roles, custom authorizers, and Amazon Cognito integration.

- **Version Management**

- Enables management of multiple API versions so different clients can access the correct version seamlessly.

- **Scalability**

- Automatically scales to handle high-volume or sudden increases in API requests without downtime.

- **Monitoring and Analytics**

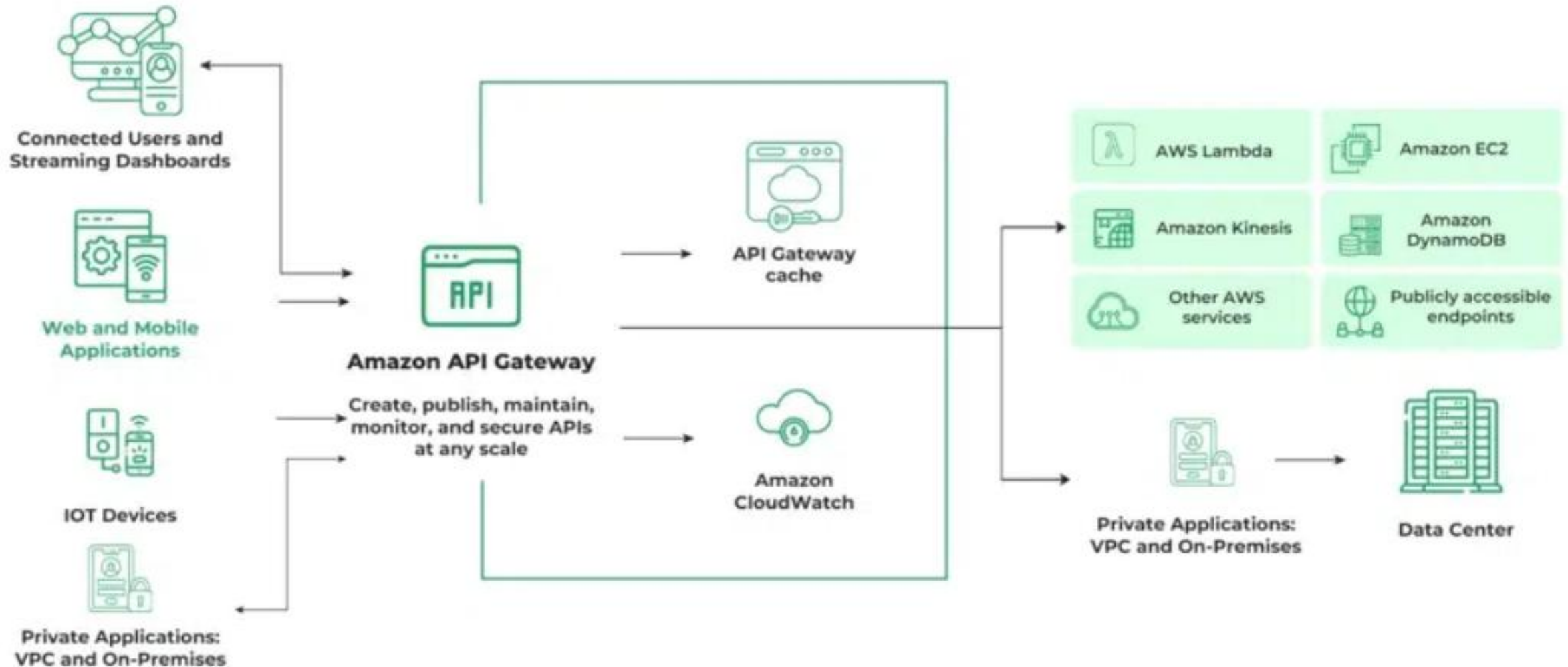
- Integrated with Amazon CloudWatch to track performance metrics, request logs, and diagnose issues effectively.

How Amazon API Gateway Works?

- Amazon API Gateway acts as an interface between client applications and backend services. When a client sends a request, the API Gateway routes it to the appropriate backend service based on the API configuration. Once the backend processes the request, the service sends a response back to the API Gateway, which then passes it to the client.
- **Simplified flow:**

Client Request → API Gateway → Backend Service (AWS Lambda, EC2, etc.) → Response → API Gateway → Client

This architecture minimizes client-side complexity while centralizing API management.



Types of APIs Supported by Amazon API Gateway

- AWS API Gateway supports multiple API types to suit different application needs.
- **REST APIs (Public and Private)**
 - Follow the REST architectural style for building web services.
 - Provide a simple, flexible way to access data without heavy processing.
- **SOAP APIs**
 - Use the SOAP protocol for structured data exchange over XML.
 - Work across different platforms and operating systems using protocols like HTTP and SMTP.
- **GraphQL APIs**
 - Use GraphQL, an open-source query language and runtime for APIs.
 - Allow clients to request exactly the data they need, improving performance and flexibility.
- **WebSocket APIs**
 - Enable real-time, two-way communication between clients and servers.
 - Useful for applications like chat systems, live dashboards, and notifications.

Amazon API Management Components

- Following are some Amazon API Management Components that you need to be familiar with to get a better understanding.
- **HTTP (HyperText Transfer Protocol) API** is an application layer protocol that helps to communicate over the World Wide Web to get the data.
- **REST (Representational State Transfer) API** takes the HTTP standards to perform operations of **GET, POST, PUT, PATCH**, and **DELETE** on the data.
- **WebSocket** is a device communication protocol that provides point-to-point system communication channels over a single TCP. It enables stateful, full-duplex communication between client and server.

Benefits Of Amazon API Gateway

- **Simple to Use:** The AWS console provides very simple UI to use API GW which make its easy to manage and monitoring APIs.
- **Scalable:** Usually Amazon web services maintains very high scalable API gate ways then only they can handle the heavy demanding workloads.
- **High Secure:** API Gateway offers several capabilities, like custom authorizers and IAM integration, to help secure APIs.
- **Exposes Backend Services:** We can expose backend services to external clients, such web applications, mobile applications, and other APIs, with the use of API gateways.

Features of Amazon API Gateway

- **API creation and deployment:** Creating different types of APIs and managing them will be very easy.
- **Authorization and access control:** With the help of [IAM roles](#), [IAM policies](#), and custom authorizers you can achieve authorization and control mechanisms that API Gateway offers for your APIs.
- **Traffic management:** Load balancing, throttling and caching are some of the services offered by the API Gateway to manage the traffic.
- **API versioning:** Based on the request you API Gateway will route the traffic to the and to route traffic to the appropriate version based on the client's request.

Pricing of Amazon API Gateways (GW)

- You pay for use only, i.e you pay for the API calls you receive and the amount of data transfer. There is an optional data caching charged at an hourly rate that varies based on the cache size you select.
- Amazon also provides free tier services for **up to 12 months**, which includes:
 - 1 million HTTP API calls
 - 1 million REST API calls
 - 1 million messages
 - 750,000 connection minutes per month

Standard pricing of Amazon API Gateway:

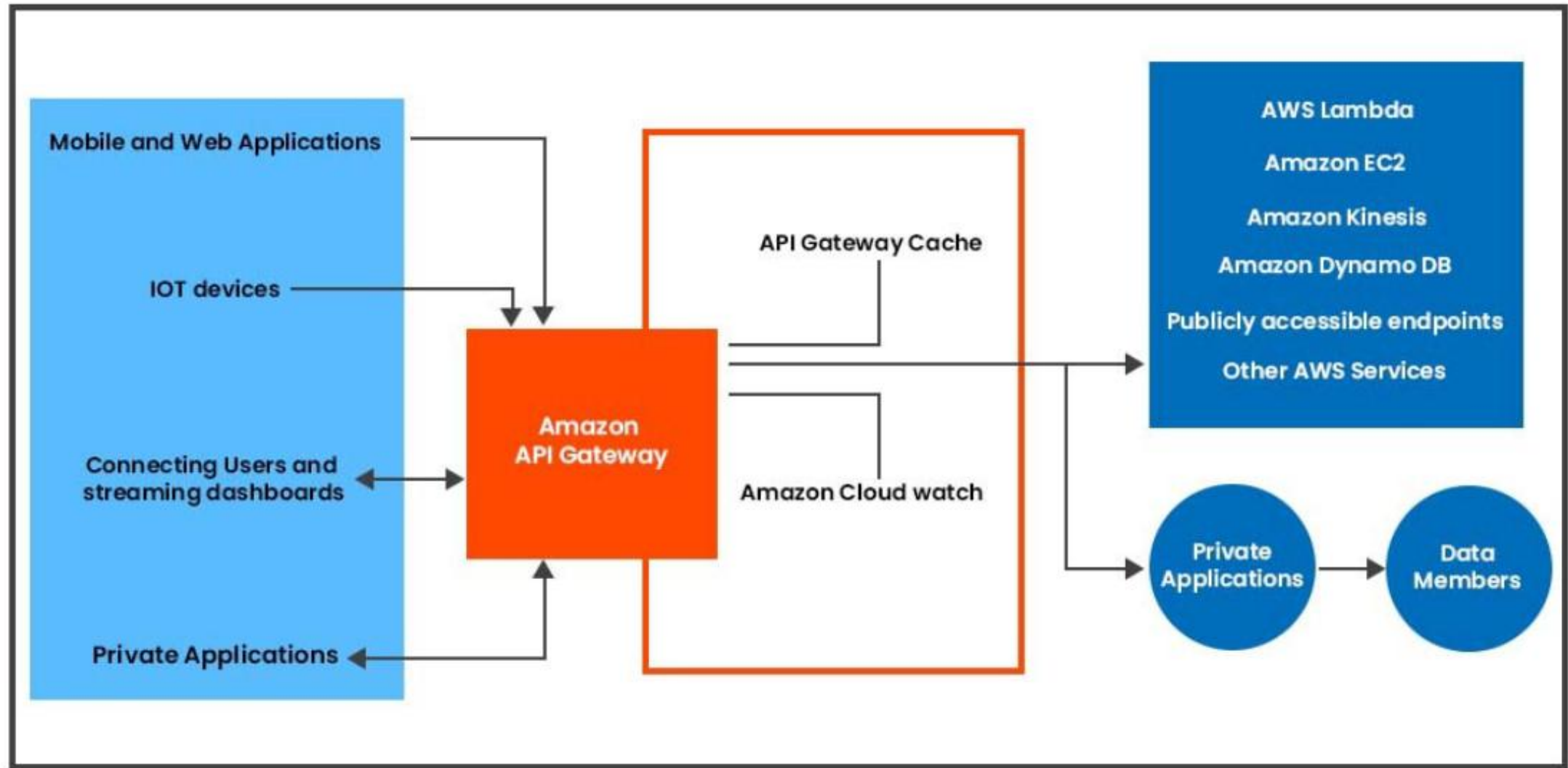
Free Tier:

- 1 million HTTP API calls
- 1 million REST API calls
- 1 million messages
- 750,000 connection minutes per month for 12 months

Pricing for the API Calls		
APIs	No. of requests per month	Price(per million)
HTTP	First 300 million	\$1.00
	300+ million	\$0.90
REST	First 333 million	\$3.50
	Next 667 million	\$2.80
	Next 19 billion	\$2.38
	Over 20 billion	\$1.51
WebSocket	First 1 billion	\$1.00
	Over 1 billion	\$0.80

Amazon API Gateway Architecture

- API Gateway provides a unified and consistent developer experience for building serverless applications on AWS.
- It acts as a gateway that connects client applications to backend services.
- Backend services can include Amazon EC2, AWS Lambda functions, or any custom web application.
- The architecture ensures secure, scalable, and efficient communication between clients and backend systems.



The entire architecture of AWS API Gateway consists of the below key components:

- **Amazon API Gateway:** It is used to create, publish, secure, maintain and monitor APIs.
- **API Gateway Cache:** Users can enable API caching to cache their endpoint responses, which can reduce the number of calls made to the endpoint and also improve the latency of API requests.
- **Amazon Cloud Watch:** It is a monitoring and observability service. It collects monitoring and operational data and visualizes it using automated dashboards, which allows users to visually monitor calls to their services.

- **Working With Amazon API Gateway**
- You can access **Amazon API Gateway** through the following tools:
- **AWS Management Console**
- **AWS SDKs**, including API Gateway V1 and V2 APIs
- **AWS Command Line Interface (CLI)**
- **AWS Tools for Windows PowerShell**

Example

- To create an HTTP API, you must first create an AWS Lambda function that will serve as the backend.
- After creating and configuring the Lambda function, you can set up an HTTP API using Amazon API Gateway.
- Once the API is created, you can test it directly from the console.
- **Steps to Get Started in the AWS Management Console:**
- Log in to your AWS account and open the AWS Management Console.
- Click on **Services** and search for **API Gateway**.

 Services

APIg

Search results for 'API'

Services (25)

Features (17)

Blogs (2,948)

Documentation (751,749)

Knowledge Articles (30)

Tutorials (15)

Events (66)

Marketplace (1,751)

Services

See all 25 results ▶

 **API Gateway** ☆

Build, Deploy and Manage APIs

Top features

[APIs](#) [Custom domain names](#) [VPC links](#)

 **CloudTrail** ☆

Track User Activity and API Usage

 **Amazon AppFlow** ☆

Amazon AppFlow integrates apps and automates data flows without code.

 **Amazon Location Service** ☆

Securely and easily add location data to applications.

Features

See all 17 results ▶

Create API

 AWS AppSync feature

S Console Home

Get insights for your AWS services with the customizable Console Home

[Learn more](#)

now

Connected to your AWS

on-the-go

AWS Console Mobile App now supports additional regions. Download the AWS Console Mobile App to your iOS or Android mobile device. [Learn more](#)

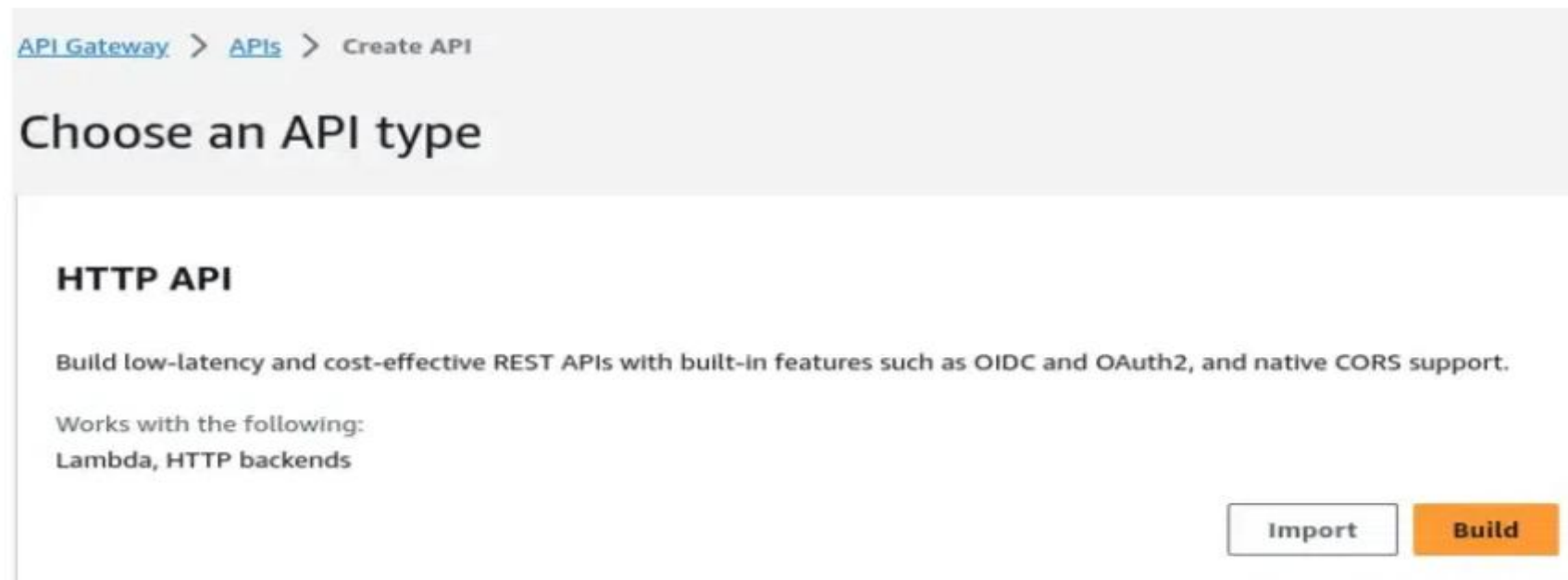
AWS

How to Create HTTP API Gateway

- Here's a step-by-step guide to creating an HTTP API Gateway:
- HTTP API is an protocol which is used to communicate between the cline and servers.

Step 1: Choose API Type

- Select the type of API that best fits your organization's requirements. For this demonstration, we will choose **HTTP API**.



Step 2: Select Integration

- Choose the backend service you want to integrate with the API. For instance, API Gateway can invoke the Lambda function you've created earlier and return the function's response to the client.

The screenshot shows the AWS API Gateway console during the 'Create an API' process. On the left, a sidebar lists four steps: Step 1 (Create an API), Step 2 (optional, Configure routes), Step 3 (optional, Define stages), and Step 4 (Review and Create). The main panel is titled 'Create an API' and contains a section 'Create and configure integrations'. This section explains that integrations specify backend services and provides examples for Lambda and HTTP integrations. Below this, it shows 'Integrations (0)' with an 'Add integration' button. At the bottom, there is a field for 'API name' with the value 'GFG-API' entered. The field has a tooltip explaining that the name is cosmetic and that the API will be referred to by its ID. At the bottom right, there are three buttons: 'Cancel', 'Review and Create', and 'Next'.

Step 1
Create an API

Step 2 - optional
[Configure routes](#)

Step 3 - optional
[Define stages](#)

Step 4
[Review and Create](#)

Create an API

Create and configure integrations

Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.

Integrations (0) [Info](#)

[Add integration](#)

API name
An HTTP API must have a name. This name is cosmetic and does not have to be unique; you will use the API's ID (generated later) to programmatically refer to this API.

GFG-API

[Cancel](#) [Review and Create](#) [Next](#)

Step 3: Define Routes and Methods

- API Gateway uses routes to expose integrations to end users. Choose the HTTP method (GET, POST, PATCH, HEAD, etc.) based on the needs of your organization and how the API will handle requests.

[API Gateway](#) > [APIs](#) > [Create API](#) > Create

Step 1
[Create an API](#)

Step 2 - optional
Configure routes

Step 3 - optional
[Define stages](#)

Step 4
[Review and Create](#)

Configure routes - *optional*

Configure routes [Info](#)

API Gateway uses routes to expose integrations to consumers of your API. Routes for HTTP APIs consist of two parts: an HTTP method and a resource path (e.g., GET /pets). You can define specific HTTP methods for your integration (GET, POST, PUT, PATCH, HEAD, OPTIONS, and DELETE) or use the ANY method to match all methods that you haven't defined on a given resource.

Method	Resource path	Integration target
Add route		

Cancel [Review and Create](#) [Previous](#) [Next](#)

Step 4: Deploy the API

- Once the setup is complete, deploy your API Gateway

[API Gateway](#) > [APIs](#) > [Create API](#) > Create

Step 1
[Create an API](#)

Step 2 - optional
[Configure routes](#)

Step 3 - optional
Define stages

Step 4
Review and Create

Define stages - optional

Configure stages [Info](#)

Stages are independently configurable environments that your API can be deployed to. You must deploy to a stage for API configuration changes to take effect, unless that stage is configured to autodeploy. By default, all HTTP APIs created through the console have a default stage named \$default. All changes that you make to your API are autodeployed to that stage. You can add stages that represent environments such as development or production.

Stage name Auto-deploy

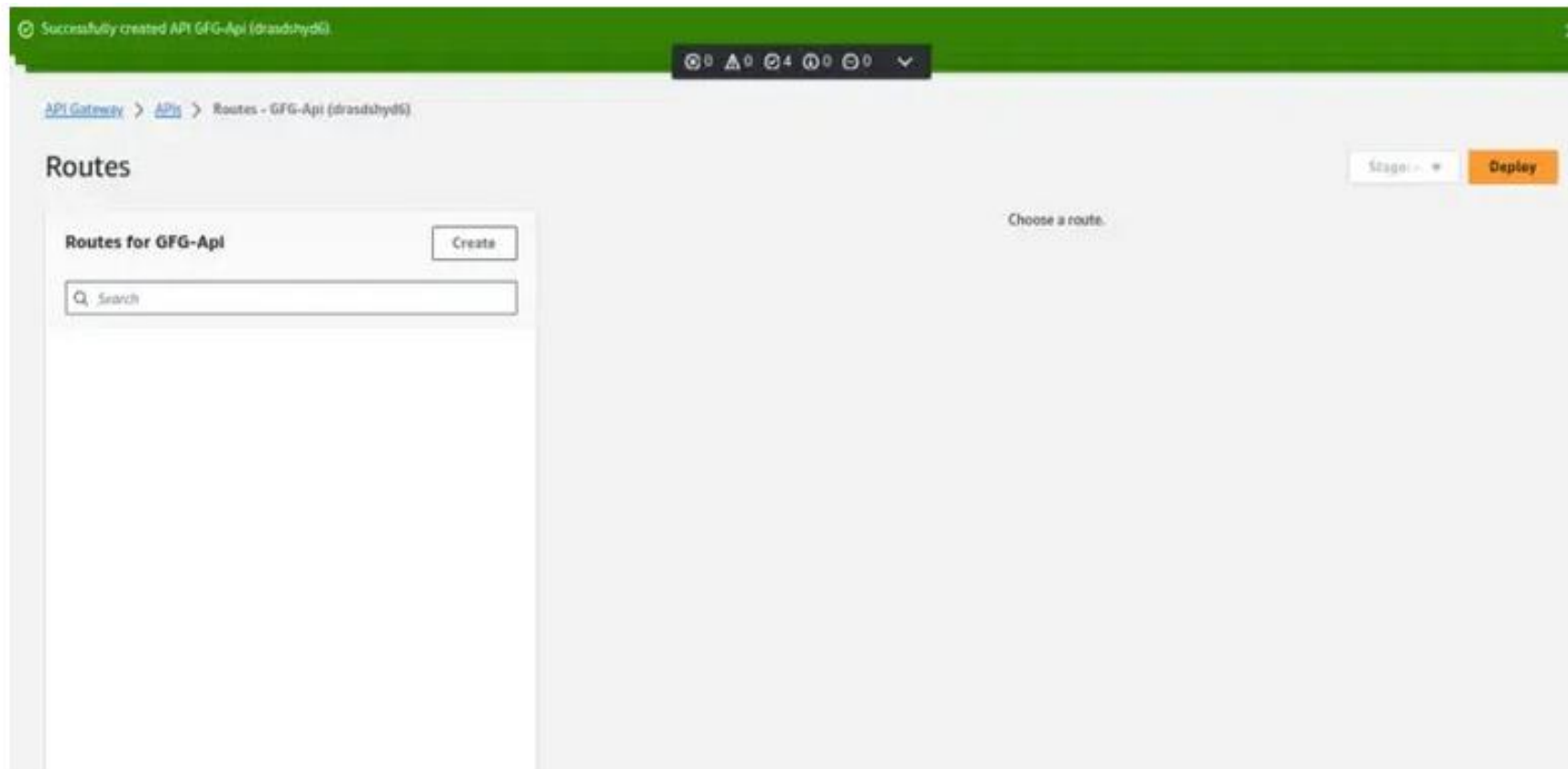
☒ Remove

Add stage

Cancel Previous Next

Step 5: Review and Create

- After reviewing all your settings and configurations, click Review and Create. Once the API is created, you can use it with the name you've assigned.



Amazon API Gateway Advantages

- Enables efficient API development by allowing multiple versions of the same API to run simultaneously, helping in quick testing and iteration.
- Provides very low latency for API requests and responses, improving overall performance.
- Offers easy performance monitoring through the API Gateway dashboard.
- Cost-efficient at scale, allowing users to reduce costs as API usage increases across regions and accounts.
- Provides flexible security controls using AWS IAM, Amazon Cognito, and custom authorizers.
- Reduces coding effort, making application development more efficient with fewer chances of errors.
- Hides and encapsulates the internal structure of backend applications from clients.
- Allows clients to interact through a single entry point instead of calling individual services directly.
- Simplifies the management and handling of multiple APIs in one centralized place.

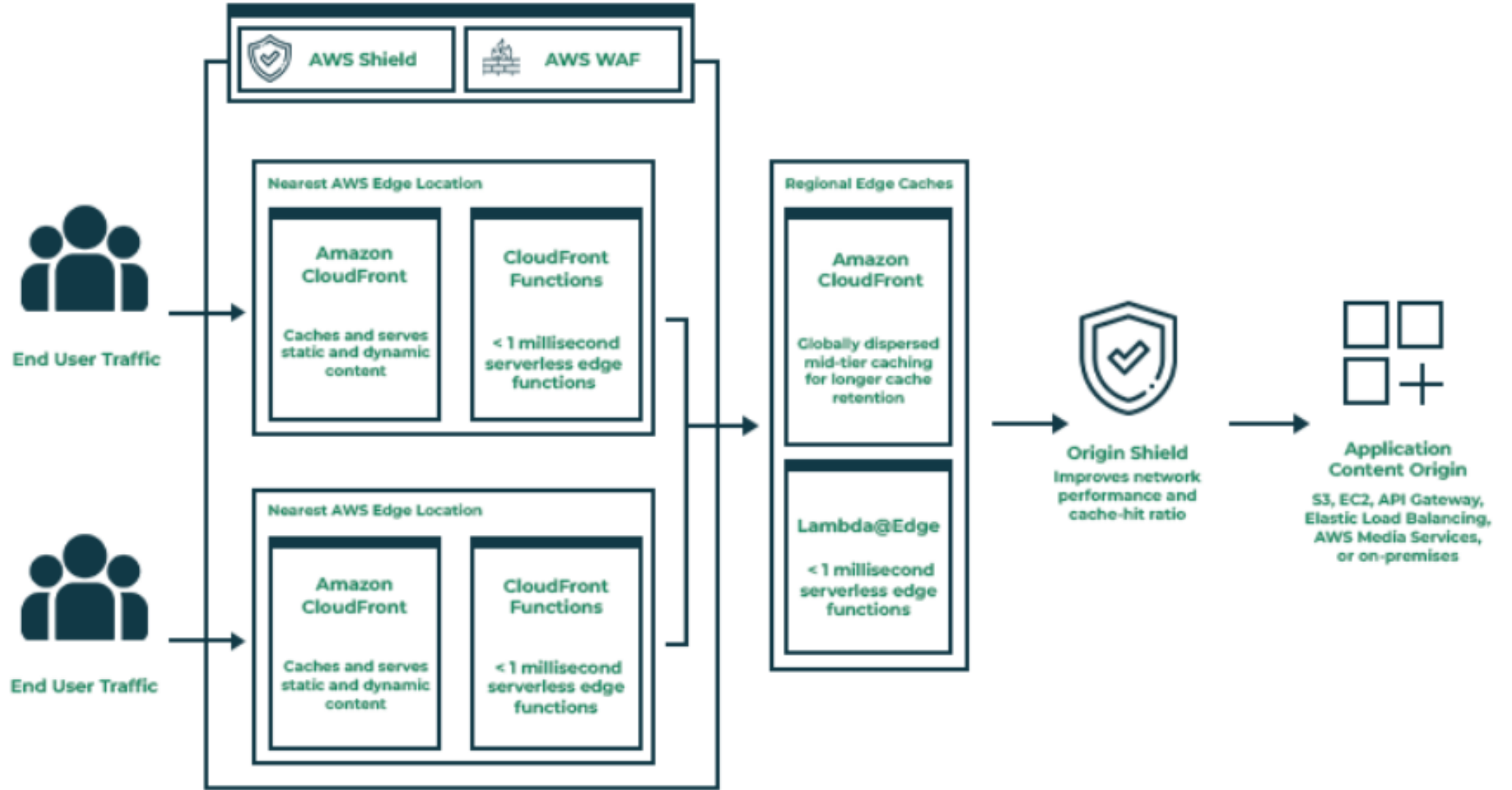
Amazon API Gateway Disadvantages

- Security implementation can be challenging, as applying security practices to every API call may become complex and make performance tracking difficult.
- API Gateway may not always be considered fully reliable due to dependency on a single gateway layer.
- Maintaining the API Gateway becomes difficult when an application grows into hundreds or thousands of microservices.
- Requires routing rules, which can create a single point of failure if not managed properly.
- Storing all API rules in one place increases the risk of complexity, especially as the number of services scales.

AWS CloudFront

AWS CloudFront Overview

- AWS CloudFront is a fast content delivery network (CDN) that speeds up the delivery of websites, videos, APIs, and applications.
- It works by storing copies of your content in multiple edge locations worldwide.
- When a user visits your site, CloudFront delivers content from the nearest edge location, reducing latency and improving load times.



What is AWS CloudFront?

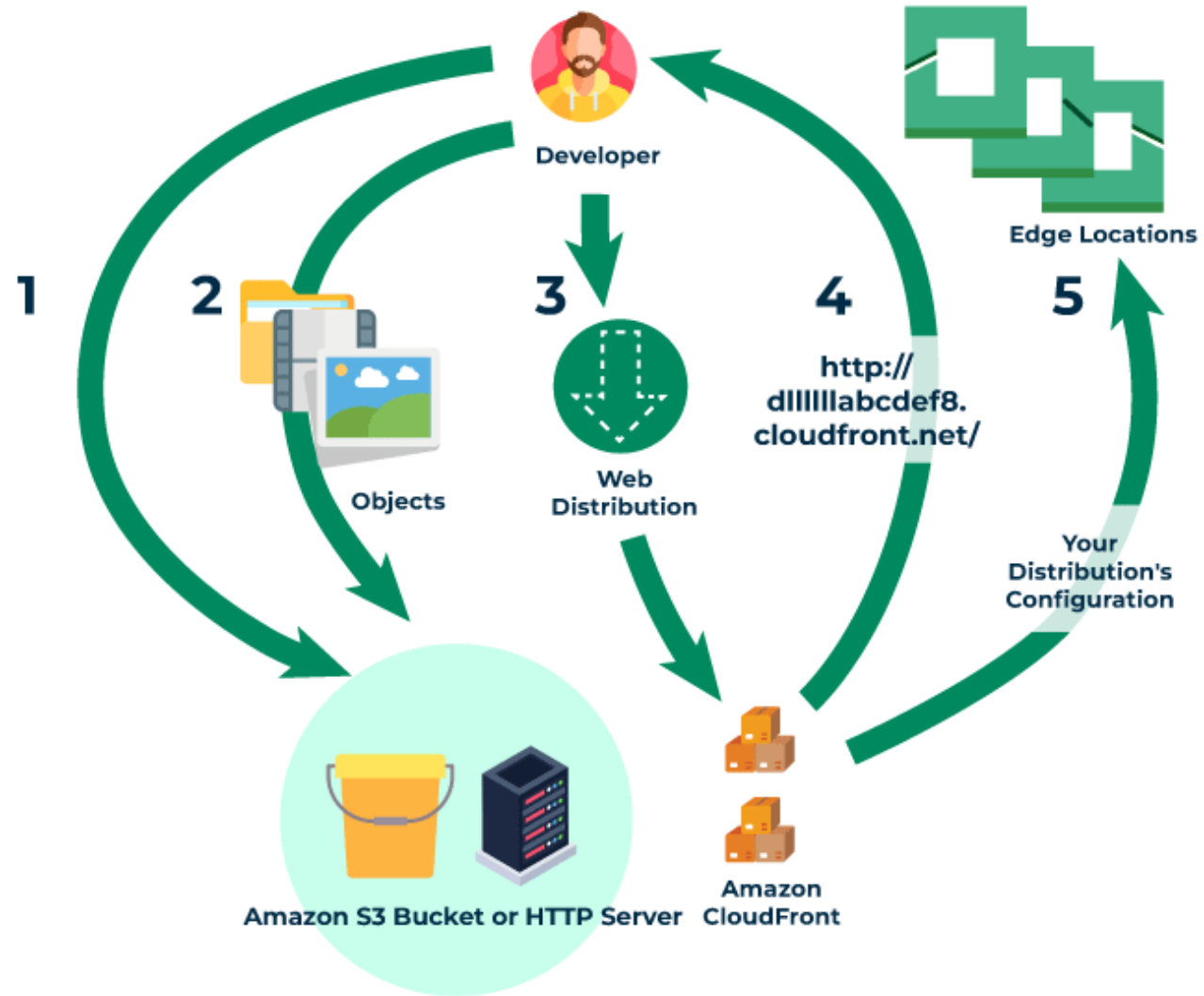
- AWS CloudFront is a global Content Delivery Network (CDN) service offered by Amazon Web Services.
- It helps deliver content—such as static files, dynamic data, videos, images, and APIs—to users quickly and efficiently across the world.
- CloudFront stores cached copies of your content in *edge locations*, which are servers placed globally for faster delivery.
- When a user makes a request, CloudFront delivers the content from the nearest edge location, reducing latency and improving performance.
- It integrates seamlessly with AWS services like Amazon S3, EC2, Lambda@Edge, and API Gateway.

Key Components of AWS CloudFront

- **Edge Locations:** Global data centers that cache content closer to users.
- **Origin Server:** The main location where your original content is stored (e.g., S3 bucket, EC2 instance, or on-premises server).
- **Distribution:** Configuration that defines how CloudFront delivers content.
 - Web Distribution (for websites, APIs, static/dynamic content)
 - RTMP Distribution (for streaming media; now deprecated)
- **Cache Behaviors:** Rules that define how content should be cached (TTL settings, cookies, query strings, etc.).
- **Signed URLs & Cookies:** Used to restrict access and protect private content.
- **Lambda@Edge:** Allows running custom logic at edge locations to modify requests/responses.

How AWS CloudFront Works

- **User Requests Content**
 - A user requests a webpage, image, video, or API response from a website or application.
- **DNS Routes the Request**
 - DNS identifies the nearest CloudFront edge location and routes the request there.
- **CloudFront Checks Cache**
 - If the content is already cached at the edge location → it is delivered immediately.
 - If not cached → CloudFront forwards the request to the origin server.
- **Origin Server Sends Content**
 - The origin (S3, EC2, or on-premises server) returns the requested content to CloudFront.
- **CloudFront Caches the Content**
 - CloudFront stores the new content in the edge location for future requests.
- **Content Delivered to the User**
 - The user receives the content quickly from the nearest edge location.
- **Faster Future Requests**
 - Next time, CloudFront delivers the cached version instantly without contacting the origin.
- **Cache Updates When Needed**
 - CloudFront checks the origin periodically. If the content has changed, it updates the cached version.



Key Features of AWS CloudFront

- **Faster Content Delivery Across the Globe**
 - CloudFront stores copies of content in many global edge locations.
 - Users receive content from the nearest location, ensuring faster loading times.
- **Seamless Integration with AWS Services**
 - Works smoothly with S3 (file storage), EC2 (web hosting), API Gateway (APIs), Route 53 (DNS), AWS WAF (security), and ELB (load balancing).
- **Built-in Security & Attack Protection**
 - Protects against DDoS attacks and malicious traffic through AWS Shield and AWS WAF.
 - Supports HTTPS and access control for secure content delivery.
- **Efficient Caching for Static & Dynamic Content**
 - Static files (images, CSS, JS) are cached and delivered instantly.
 - Dynamic content (APIs, personalized pages) is optimized for low-latency delivery.
- **Budget-Friendly & Scalable**
 - Pay only for what you use.
 - AWS Free Tier includes **1 TB** of data transfer per month.
- **Customizable with Lambda@Edge**
 - Modify request/response behavior at edge locations (e.g., header changes, redirects, A/B testing).
- **Low Latency & High Performance**
 - CloudFront finds the fastest route to deliver content, providing smoother user experiences.

AWS CloudFront & AWS WAF Integration

What is AWS WAF?

- A security service that protects web applications from threats like SQL injection and XSS.
- Filters harmful requests and allows you to set custom security rules.

How CloudFront Works with WAF

- You create **Web ACLs** (Access Control Lists) containing security rules.
- When a user requests content, CloudFront routes it to the nearest edge location.
- The Web ACL checks the request:
 - **If allowed:** The request goes to the origin.
 - **If blocked:** The request is stopped at the edge.

What is a Web ACL?

- A set of rules used to control traffic based on:
- IP addresses
- Request headers
- Query strings or request body
- Geographic locations

AWS CloudFront Use Cases

- **Delivering Static Web Content**

- Static files (HTML, CSS, JS, images) are cached at edge locations for fast delivery.

- **Streaming Media**

- Integrates with S3 to stream video and audio without buffering.

- **Serving Dynamic Content**

- Lambda@Edge helps generate and deliver dynamic data efficiently.

- **Global Content Delivery**

- Content is cached near users around the world, reducing latency and improving accessibility.

- **Key Benefits of AWS CloudFront**

- No mandatory upfront investment
- Reduced operational costs
- Highly scalable and resilient
- Easy global access
- Lower maintenance effort and reduced business risks

AWS CloudFront Pricing

- The table below provides a detailed knowledge of Cost of CloudFront:

Pricing Component	Description	Cost
Data Transfer Out	Data delivered from CloudFront to the internet.	Starting at \$0.085 per GB for the first 10 TB/month in the U.S., Mexico, and Canada.
HTTP/HTTPS Requests	Number of requests processed by CloudFront.	\$0.0075 per 10,000 HTTP requests; \$0.0100 per 10,000 HTTPS requests in the U.S. region.
Invalidation Requests	Removing cached objects before expiration.	First 1,000 paths free each month; \$0.005 per path thereafter.
Real-Time Log Requests	Detailed logging of CloudFront requests.	\$0.01 per 1,000,000 log lines.
Origin Shield Requests	Additional caching layer to reduce origin load.	\$0.0075 per 10,000 requests in the U.S. region.

Companies using AWS CloudFront

The following are list of companies using AWS CloudFront:

Company	Use Case
United States Department of Defense	Secure content distribution across its extensive network.
Walmart	Enhances performance and reliability of its online retail platform.
Amazon	Accelerates content delivery for its e-commerce operations.
Netflix	Streams high-quality video content globally with low latency.
Hulu	Delivers streaming media content efficiently to its users.
Disney	Supports Disney+ with scalable and secure content delivery.
BMW	Enhances vehicle connectivity and digital services via CloudFront.
Unilever	Powers digital marketing and global campaign deployment.
McDonald's	Optimizes customer experience with machine learning and analytics.
Capital One	Supports banking services with high-security cloud solutions.

Amazon Virtual Private Cloud

What is a VPC (Virtual Private Cloud)?

- A VPC is an on-demand, configurable pool of cloud resources within a public cloud environment.
- It provides a **logically isolated** and **secure** section of the cloud, similar to a private data center.
- Organizations can deploy applications and store data in a private virtual network that looks and works like an on-premises setup but benefits from the cloud's scalability and flexibility.
- A VPC ensures your resources stay separate from other cloud tenants, giving you full control over networking, security, and access.

Benefits of Using a VPC

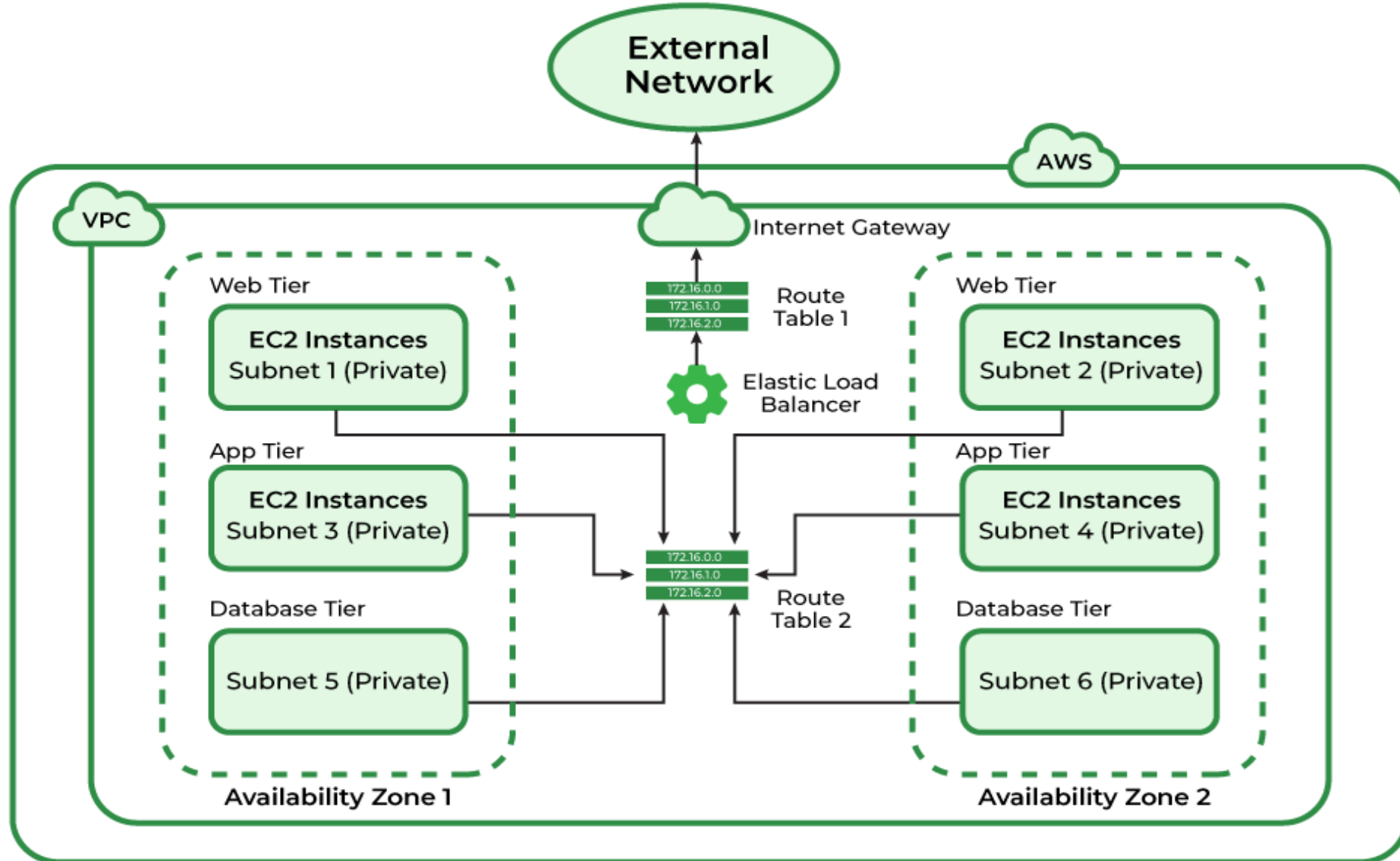
- **Security**
 - VPC ensures strong logical isolation of workloads.
 - Provides strict access controls to keep resources safe from unauthorized access.
 - Keeps your environment isolated from other customers.
- **Agility & Scalability**
 - Allows organizations to quickly deploy, modify, or scale resources.
 - No need to purchase or install physical hardware.
 - Network size and configuration can be changed anytime.
- **High Availability**
 - Uses the cloud provider's fault-tolerant infrastructure.
 - Supports deployment across multiple Availability Zones to ensure redundancy.
 - Improves uptime and reliability of applications.
- **Cost Efficiency**
 - Offers the features of a private data center without high upfront investment.
 - Reduces hardware, maintenance, and labor costs.
 - Pay-as-you-go pricing helps optimize expenses.

Example

- **Office Building** → VPC (private and secure space)
- **Departments (Editorial, HR, Development)** → Subnets (public or private)
- **Security Guards** → Security Groups & Firewalls (control who enters or leaves)
- **Internet Connection** → Internet Gateway (allows public access when required)
- **Private Tunnels to Partners** → VPN / Direct Connect (secure external connections)

Amazon VPC Architecture Overview

- A VPC is built using multiple components like **Gateways, Subnets, Load Balancers**, and more.
- These resources combine to create a controlled and isolated virtual environment.
- Networking is managed using **route tables** that connect different subnets.
- Multiple layers of security—such as NACLs and Security Groups—protect the network.
- Public and private subnets allow separation of externally accessible and internal resources.



Core Components of Amazon VPC

- The following are the key components of Amazon VPC:

1. VPC (Virtual Private Cloud)

- A logically isolated section of the AWS cloud where resources can be launched securely.
- Defined using an IPv4 CIDR block (e.g., **10.0.0.0/16**) providing up to **65,536 IP addresses**.
- Functions like an on-premises data center but with AWS scalability, reliability, and elasticity.

2. Subnets

- A subdivision of a VPC's IP range and exists in **one Availability Zone**.
- **Purpose:**
 - *Segmentation*: Organizes and isolates resources.
 - *High Availability*: Distributing subnets across AZs ensures fault tolerance.
- **Types:**
 - **Public Subnets** – route traffic through an Internet Gateway (IGW).
 - **Private Subnets** – isolated from direct internet access; use NAT Gateway for outbound traffic.

3. Route Tables

- Each VPC has an implicit virtual router that uses **Route Tables** to direct traffic.
- Every subnet must be associated with one route table.
- A route includes:
 - **Destination CIDR** (traffic target)
 - **Target** (IGW, NAT Gateway, instance, etc.)
- Determines whether traffic stays in the VPC or goes outside.

4. Network ACLs (NACLs)

- Stateless firewalls operating at the **subnet level**.
- Default NACL provided by AWS; modifiable but not deletable.
- Require separate rules for inbound and outbound traffic.
- Ideal for broad subnet-level restrictions like blocking specific IP ranges.

5. Internet Gateway (IGW)

- A horizontally scaled, redundant component enabling VPC resources to access the internet.
- Attached to a VPC to allow bidirectional communication.
- A public subnet must have a route like **0.0.0.0/0 → IGW** for internet access.
- Only **one IGW** can be attached to a VPC at a time.

6. Network Address Translation (NAT)

- **NAT Gateways** allow instances in private subnets to access the internet **outbound only**.
- Prevents unsolicited inbound traffic.
- Commonly placed in a **public subnet**.
- Ensures private instances can download updates, access APIs, etc., without exposing them.

7. Security Groups

- Stateful firewalls applied at the **instance level**.
- Allow inbound/outbound rules based on IP, port, and protocol.
- Return traffic for an allowed outbound request is automatically permitted.
- Provide granular control for EC2, RDS, Lambda functions, and other VPC resources.

8. CIDR (Classless Inter-Domain Routing)

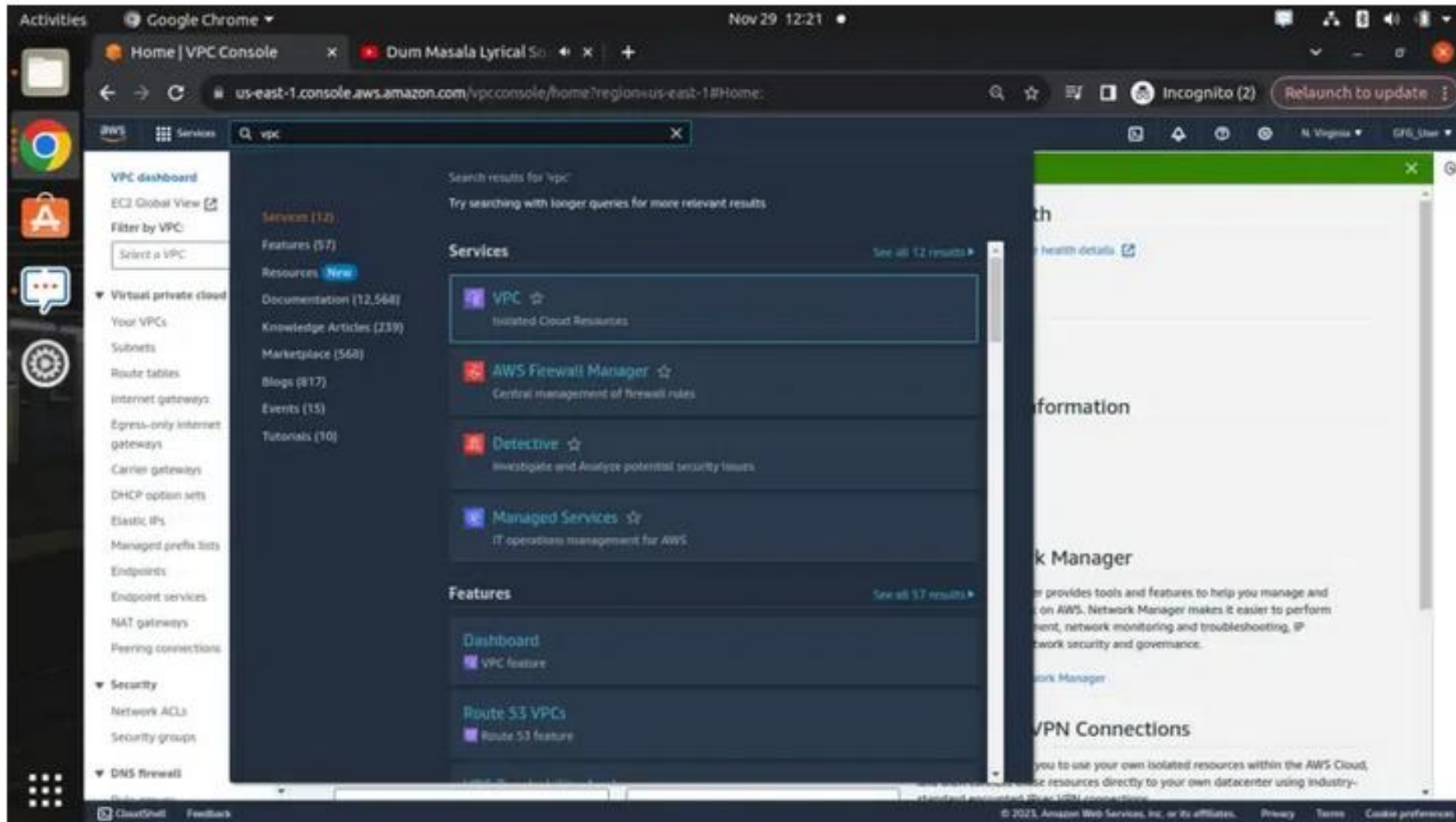
- Defines the IP addressing structure of a VPC or subnet (e.g., **10.0.0.0/16**).
- Supports flexible allocation and subdivision of private IP ranges per **RFC 1918**, including:
 - **10.0.0.0/8**
 - **172.16.0.0/12**
 - **192.168.0.0/16**

Use Cases of Amazon VPC

- Hosting public applications (web servers, ALBs) in public subnets.
- Deploying multi-tier architectures (app servers + databases) in private subnets.
- Secure inter-VPC connectivity via peering or AWS Transit Gateway.
- Enforcing security and compliance through isolation, monitoring, and access control (e.g., VPC Flow Logs, Security Groups, NACLs).

Amazon VPC Working – Steps

Step 1: Log in to the AWS Management Console and navigate to the VPC Dashboard.



Step 2: After navigating to the AWS VPC now click on create VPC.

You successfully deleted vpc-098d155bcac43e21e / GFG-VPC

Create VPC

Launch EC2 Instances

Note: Your Instances will launch in the US East region.

Resources by Region

Refresh Resources

You are using the following Amazon VPC resources

VPCs See all regions US East 1	NAT Gateways See all regions US East 0
Subnets See all regions US East 6	VPC Peering Connections See all regions US East 0
Route Tables See all regions US East 1	Network ACLs See all regions US East 1
Internet Gateways See all regions US East 1	Security Groups See all regions US East 1
Egress-only Internet Gateways See all regions US East 0	Customer Gateways See all regions US East 0
DHCP option sets US East 1	Virtual Private Gateways US East 0

Step 3: Configure all the details required to create as shown in the image below. Some of the most required settings to configure VPC are as follows

- Name of the Network.
- IPv4 CIDR.
- And tags of VPC after that click on create VPC.

Create VPC [Info](#)

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

GFG-VPC

IPv4 CIDR block [Info](#)
☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR
10.0.0.0/24
CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)
☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Tenancy [Info](#)
Default

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Q, Name	X	
Q, GFG-VPC	X	Remove tag

[Add tag](#)
You can add 40 more tags.

[Cancel](#) [Create VPC](#)

Step 4: Virtual Private Cloud created successfully with the required settings

The screenshot displays the AWS VPC console interface in a Google Chrome browser. The address bar shows the URL: `us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#VpcDetails:VpcId=vpc-098d155bcac43e21e`. A green notification banner at the top states: "You successfully created vpc-098d155bcac43e21e / GFG-VPC".

The main content area shows the details for the VPC `vpc-098d155bcac43e21e / GFG-VPC`. The details are organized into a table:

Details			
VPC ID	State	DNS hostnames	DNS resolution
vpc-098d155bcac43e21e	Available	Disabled	Enabled
Terminating	DHCP options set	Main route table	Main network ACL
Default	dhcp-0a1341a4aeb0b6754	vrtb-018d06a1b5e7bdc130	acl-05a6c70ff5a00754
Default VPC	IPv4 CIDR	IPv4 pool	IPv4 CIDR / Network border group
No	10.0.0.0/24	--	--
Network Address Usage metrics	Route 53 Resolver DNS Firewall rule groups	Creation ID	
Disabled	--	436523482014	

Below the details table, there are tabs for "Resource map", "CIDRs", "Flow logs", "Tags", and "Integrations". The "Resource map" tab is selected, showing a visual representation of the VPC resources. It includes a box for the VPC (GFG-VPC), a box for Subnets (0), a box for Route tables (1) (vrtb-018d06a1b5e7bdc130), and a box for Network connections (0). A feedback prompt at the bottom asks: "Was the resource map helpful today? Give us feedback as often as possible. We are improving continually."

Step 5: Check the VPC dashboard whether the VPC created is available to use as shown in the image below GFG-VPC.

☐	Name ▾	VPC ID ▾	State ▾	IPv4 CIDR ▾	IPv6 CIDR ▾
☐	-	vpc-08d4ac89e9417b4bb	✔ Available	172.31.0.0/16	-
☐	GFG-VPC	vpc-098d155bcac43e21e	✔ Available	10.0.0.0/24	-

AWS VPC Console

- To create and manage a Virtual Private Cloud (VPC) in AWS, follow these steps:
- **Log in** to your AWS account.
- Once inside the **AWS Management Console**, click on the “**Services**” menu at the top.
- From the list of categories, navigate to “**Networking & Content Delivery**”.
- Select “**VPC**” from the options provided.
- After selecting VPC, you will be redirected to the VPC dashboard. On the left-hand side, the **navigation pane** displays various options and services related to VPC management.

☐ New VPC Experience
Tell us what you think

Customer Gateways

Virtual Private
Gateways

Site-to-Site VPN
Connections

Client VPN Endpoints

▼ **TRANSIT
GATEWAYS**

Transit Gateways

Transit Gateway
Attachments

Transit Gateway Route
Tables

Transit Gateway
Multicast

Network Manager

▼ **TRAFFIC
MIRRORING**

Mirror Sessions *New*

Mirror Targets *New*

Mirror Filters *New*

Settings *New*

Launch VPC Wizard

Launch EC2 Instances

Note: Your instances will launch in the Asia Pacific (Mumbai) region.

Resources by Region [Refresh Resources](#)

You are using the following Amazon VPC resources

VPCs

Mumbai 1

[See all regions ▼](#)

NAT Gateways

Mumbai 0

[See all regions ▼](#)

Subnets

Mumbai 3

[See all regions ▼](#)

VPC Peering Connections

Mumbai 0

[See all regions ▼](#)

Route Tables

Mumbai 1

[See all regions ▼](#)

Network ACLs

Mumbai 1

[See all regions ▼](#)

Internet Gateways

Mumbai 1

[See all regions ▼](#)

Security Groups

Mumbai 4

[See all regions ▼](#)

Egress-only Internet Gateways

Mumbai 0

[See all regions ▼](#)

Customer Gateways

Mumbai 0

[See all regions ▼](#)

DHCP options sets

Mumbai 1

[See all regions ▼](#)

Virtual Private Gateways

Mumbai 0

[See all regions ▼](#)

Demonstrate the working principles of Amazon VPC (Virtual Private Cloud) and explain the VPC subnets and Route tables.

INTRODUCTION

- Amazon VPC creates a **logically isolated network** in the AWS Cloud.
- Provides **full control** over virtual networking.
- **Key features are:**
 - ***IP Address Ranges***: Define IPv4 CIDR blocks for the VPC.
 - ***Subnet Creation***: Logical divisions within the VPC.
 - ***Route Tables and NAT configuration***: Manage traffic flow within and outside the VPC.
 - ***Security***: Utilize security groups and Network Access Control Lists (NACLs) for instance-level and subnet-level protection.

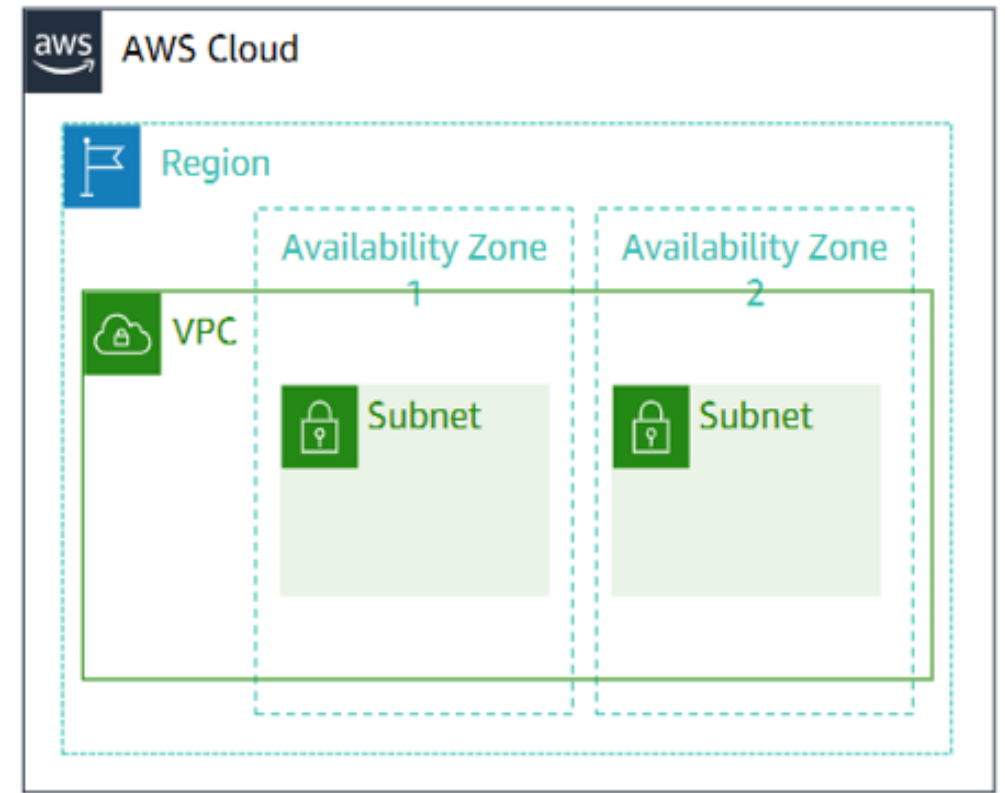
WORKING PRINCIPLES

VPC:

- Logically isolated from other VPCs.
- Dedicated to our AWS account.
- Belongs to a single AWS Region and can span multiple AZ.

Subnets:

- Range of IP addresses that divide a VPC.
- Belongs to a single AZ.
- Classified as Public or Private.



VPC SUBNET

- A **subnet** is a logical subdivision of an IP network.
- Dividing a network into smaller networks is called **subnetting**.
- **Subnetting** reduces network load and improves security.
- An **IP address** has two parts:
- **Network address** → common for all IPs in the group.

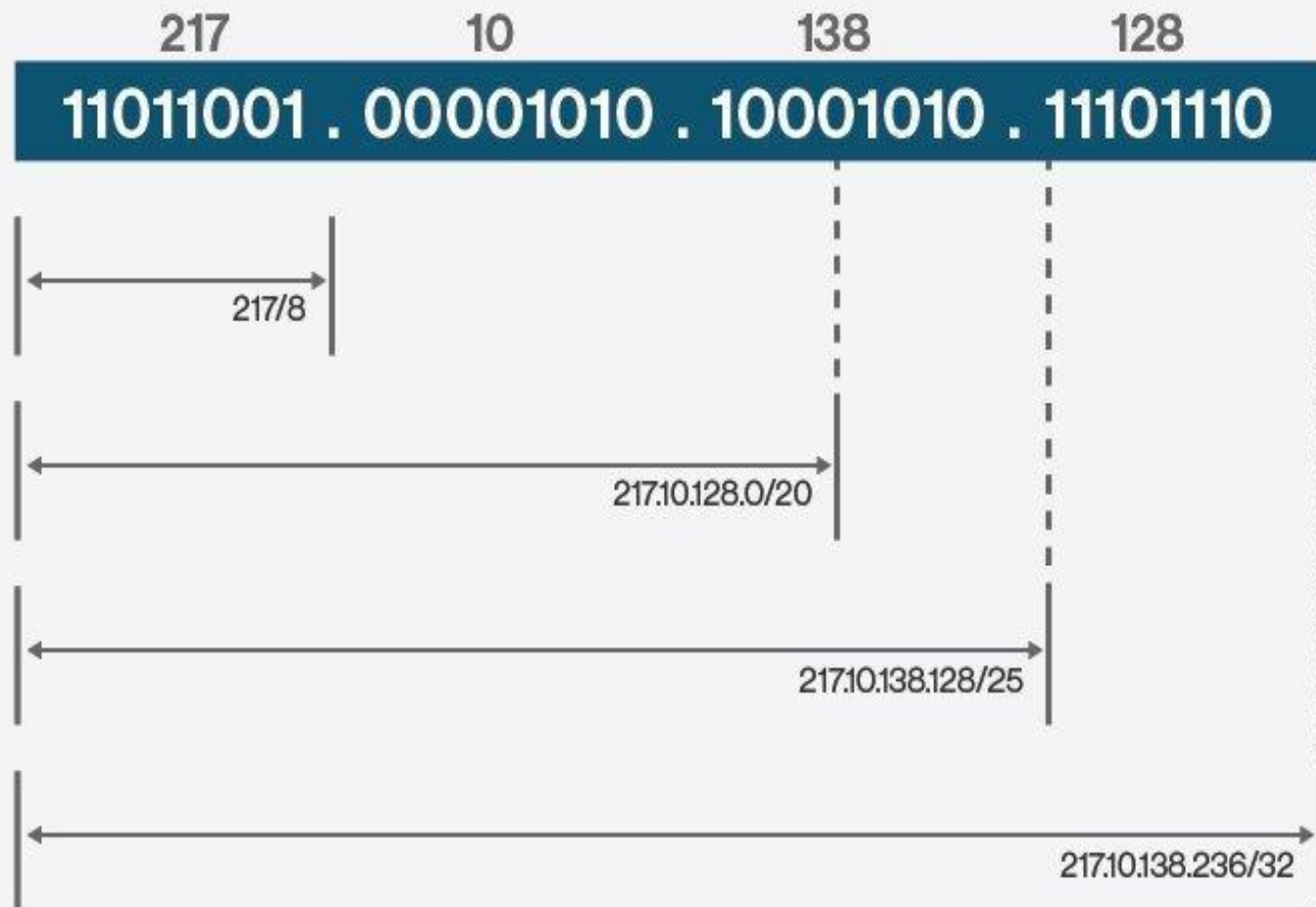
- **Host address** → unique for each device in the group.
- A **network address** = group address.
- A **host address** = node address.
- An **IP group** is called an **IP subnet**.
- **IP address** and **Subnet Mask** are both **32 bits long**.
- They are divided into **4 parts** (e.g., 125.3.2.1).
- A **Subnet Mask** assigns one bit for each IP bit (e.g., 255.255.255.0).
- **Subnet Mask bits** can be:
 - **ON (1)** → Network bit in the IP address.
 - **OFF (0)** → Host bit in the IP address.

Examples:

- IP: **10.10.10.10** → Subnet Mask: **255.0.0.0**
- IP: **171.154.12.5** → Subnet Mask: **255.255.0.0**
- IP: **192.163.1.1** → Subnet Mask: **255.255.255.0**
- If the subnet mask is: **255.255.0.0** has 8-bits for subnet and 16-bits for Host.
- 8-bits accommodate $2^8 = 256$ subnets and $2^{16} = 64000$ Hosts.

CIDR - Classless Inter-Domain Routing

- Uses **VLSM (Variable-Length Subnet Mask)** to adjust Network/Host bit ratio.
- Allocates IP addresses and routes IP packets.
- Removes fixed IPv4 class system (Class A, B, C).
- Improves **IP address distribution efficiency**.
- **CIDR address format** has two parts:
 - **Prefix** → represents the Network address (like an IP).
 - **Suffix** → total number of bits in the address.
- **Ex: 192.168.129.23/17**
- **IPv4 addresses support a maximum of 32-bits.**
- **IPv6 addresses support a maximum of 128-bits.**



Source: Technology Training Ltd.

ROUTE TABLES

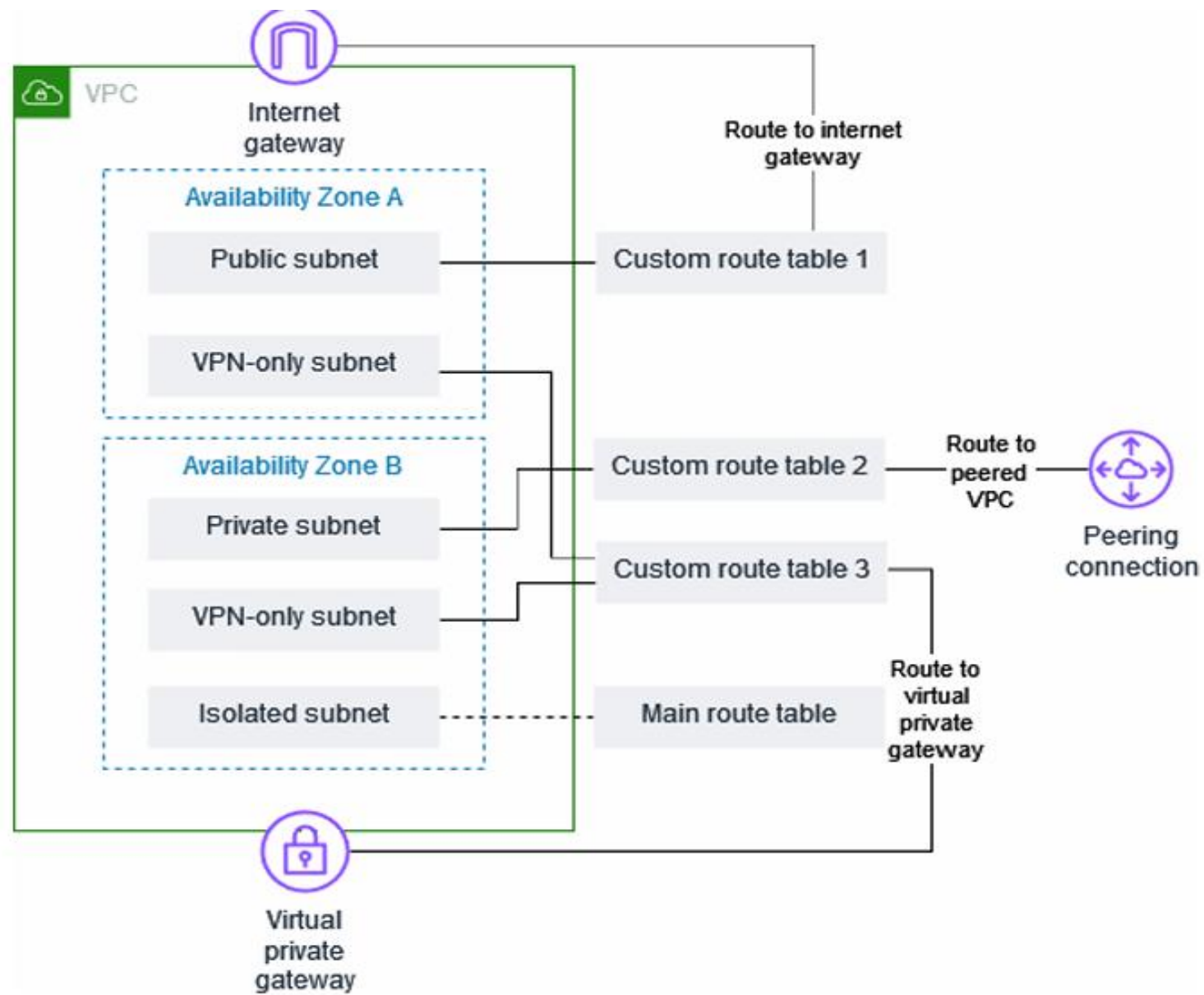
- A **route table** has rules to control subnet traffic.
- Each **route** has a **destination** and a **target**.
- By default, it includes a **local route** for communication inside the VPC.
- Each **subnet** must be linked to **one route table**.

Main (Default) Route Table

Destination	Target
10.0.0.0/16	local

VPC CIDR block

- **Destination** = target CIDR block.
- **Target** = resource handling the traffic.
- **Public subnets** use an **Internet Gateway**.
- **Private subnets** use a **NAT Gateway** for internet access.



Key Concepts:

- **Main Route Table:** Default route table in a VPC; used by subnets not linked to another table.
- **Custom Route Table:** A route table created by the user.
- **Destination:** IP range where traffic should go (e.g., 172.16.0.0/12).
- **Target:** Gateway or resource handling the traffic (e.g., Internet Gateway).
- **Local Route:** Default route inside the VPC (exists for both IPv4 and IPv6).
- **Route Table Association:** Links a route table with a subnet, internet gateway, or virtual private gateway (VPG).
- **Subnet Route Table:** Route table connected to a subnet.
- **Propagation:** Automatically adds/removes VPN routes when a VPG is attached.
- **Gateway Route Table:** Route table linked to an Internet Gateway or VPG.
- **Edge Association:** Used to route inbound VPC traffic to an appliance.
- **Transit Gateway Route Table:** Route table linked to a transit gateway.
- **Local Gateway Route Table:** Route table linked to an Outposts local gateway.

Example VPC with Route Tables

- The VPC has **5 subnets**.
- It includes **1 main route table** and **3 custom route tables**.
- All **4 route tables** have **local routes**.
- **Custom route table-1** → route to **Internet Gateway**, linked to **public subnet in AZ-A**.
- **Custom route table-2** → route to **peered VPC**, linked to **private subnet in AZ-B**.
- **Custom route table-3** → route to **Virtual Private Gateway**, linked to **VPN-only subnets** in both AZs.

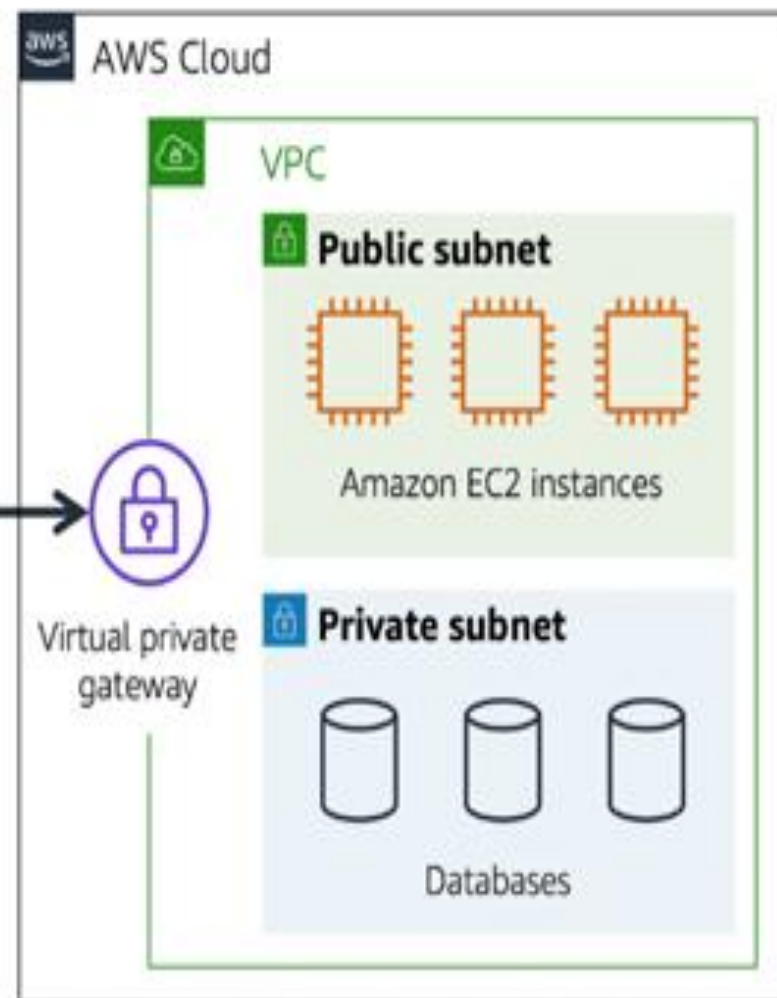
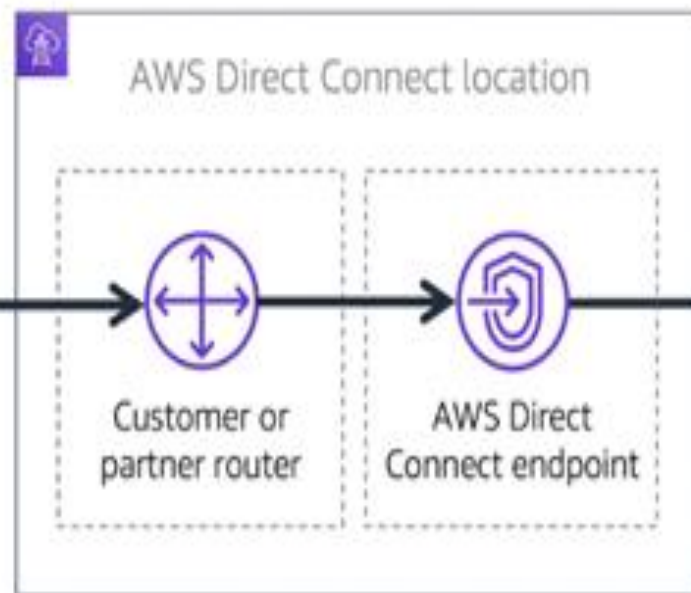
AWS Direct Connect

AWS Direct Connect

- **AWS Direct Connect** allows you to establish a **dedicated, private connection** between your on-premises data center and your AWS VPC.
- This private link **does not use the public internet**, offering greater security, consistent performance, and lower latency.
- It is ideal when you need **high bandwidth, stable connectivity**, and **secure communication** for enterprise workloads.

Analogy

- Imagine an **apartment building** directly connected to a **coffee shop** with a **private hallway**.
- Only the apartment residents can use this hallway—no one else.
- This private hallway represents **AWS Direct Connect**.
- Residents (your data center users) reach the coffee shop (AWS resources) **without using public roads** (public internet).
- Result: **Faster, safer, and exclusive access.**



- The **private connection** provided by AWS Direct Connect helps **reduce network costs** by avoiding data transfer over the public internet.
- It allows you to **increase available bandwidth**, supporting large-scale data transfers and high-performance workloads.
- Direct Connect helps organizations meet **strict regulatory and compliance requirements** by providing a secure, dedicated, non-internet-based connection.
- It also helps avoid **potential bandwidth bottlenecks** or fluctuations that commonly occur on public internet connections.
- A **single VPC can have multiple types of gateways**, such as:
 - Internet Gateway
 - NAT Gateway
 - Virtual Private Gateway (AWS VPN)
 - Direct Connect Gateway
- These gateways support different types of resources and use cases, even if they all exist **within the same VPC** but reside in different subnets.

AWS Networking and Content Delivery

Content Delivery Network (CDN)

- **AWS Networking and Content Delivery** is a collection of cloud services that help you **connect**, **secure**, and **speed up** your applications and data on AWS. These services ensure that your applications communicate efficiently, safely, and with minimal latency across the globe.

Benefits of CDN

The following are the key benefits of CDN:

1. Improved Website Performance

- CDNs reduce latency by caching content at edge locations close to users, resulting in faster load times and a smoother browsing experience. This is crucial for maintaining user engagement and satisfaction.

2. Enhanced Reliability

- CDNs distribute traffic across multiple servers, ensuring that even if one server goes down, others can handle the load. This redundancy enhances the availability and reliability of websites and online services.

3. Scalability

- CDNs can handle sudden spikes in traffic by distributing the load across their network. This scalability is essential for websites that experience variable traffic patterns, such as during product launches or viral content.

4. Security

- CDNs offer protection against DDoS attacks, provide secure data transmission through [SSL/TLS](#), and can include additional security features like web application firewalls (WAF) to safeguard content and user data.

5. Cost Efficiency

- By offloading traffic from the origin server and reducing bandwidth consumption, CDNs can help lower infrastructure and operational costs. They also minimize the need for additional server capacity to handle peak loads.

CloudFront v/s Other CDN

The following table shows the key differences between CloudFront and other CDN:

Aspect	CloudFront	CDN
Service Provider	Amazon Web Services (AWS)	Various providers like Akamai , Cloudflare
Integration	Tightly integrated with AWS services	Compatible with various hosting environments
Customization	Offers extensive customization options	Provides basic to advanced customization

- AWS Networking and Content Delivery services help with:
- **Building private networks** inside AWS
- **Connecting on-premises data centers** to AWS
- **Routing traffic** between users and applications
- **Delivering content globally at high speed**
- **Protecting applications** from attacks and unauthorized access

Key Purposes

- **1. Connectivity**
- Connect your AWS resources securely and efficiently using:
- **VPC (Virtual Private Cloud)**
- **VPN**
- **AWS Direct Connect**

2. Traffic Management

- Control how users reach your applications using:
- **Elastic Load Balancing (ELB)**
- **Amazon Route 53 (DNS)**

3. Content Delivery

- Deliver content like images, videos, and APIs faster through:
- **Amazon CloudFront (CDN)**

4. Security

- Protect network traffic and applications using:
- **AWS WAF (Web Application Firewall)**
- **AWS Shield (DDoS protection)**
- **Security Groups & NACLs**